AgenTekki: The MCP Tax Agent Project (v2.0)

1. Project Overview

AgenTekki is an agentic AI exercise where students build an autonomous system to file a Zurich tax return. The agent must process a folder of text-based documents, navigate a simplified 6-page tax portal clone, consult tax guides for rules, and interact with a simulated human taxpayer to resolve ambiguities.

The core technical challenge is **UI Auto-Discovery**. Unlike a scripted bot, this agent does not have hardcoded field IDs. It must use the **Model Context Protocol (MCP)** to "scan" the page, identify relevant input fields by their labels or roles, and decide on actions dynamically.

2. Architecture: The Dual-Path Strategy

The project is built to run in two distinct environments using the same agent logic:

1. **Google Colab (Evaluation Mode):** The UI is rendered as a cell output (HTML/JS). A Python bridge handles the MCP communication between the browser and the agent loop.
2. **Standalone (Production Mode):** A standard MCP Server using **Playwright** to drive a headless browser. This allows the agent to be connected to external clients like Claude Desktop or a CLI.

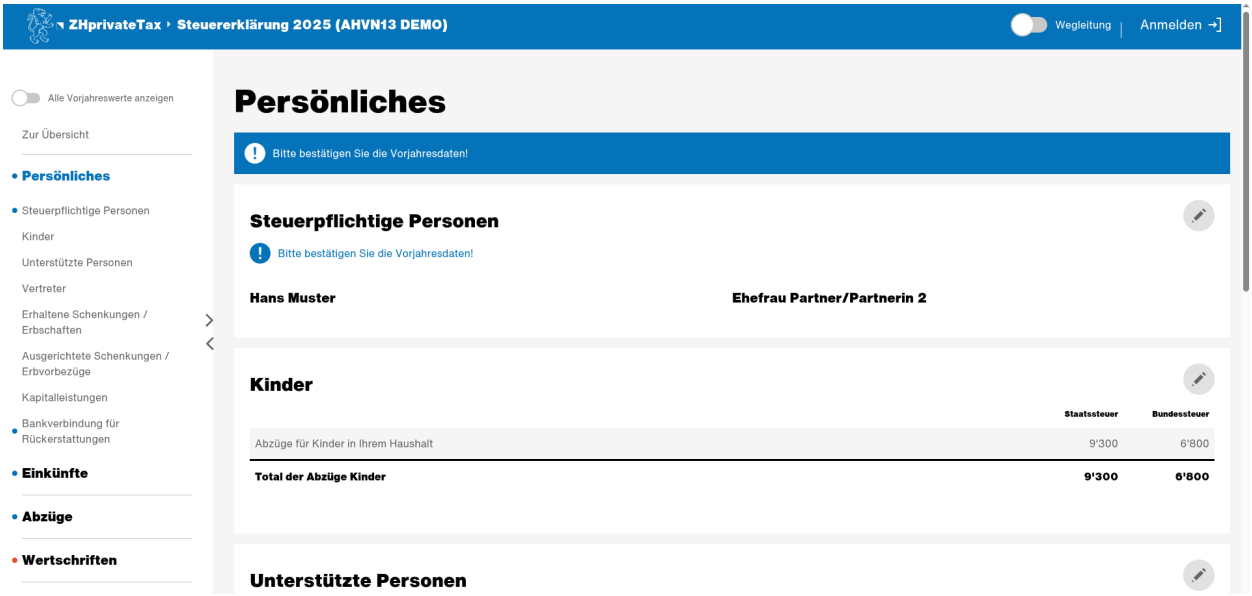
3. The Environment: ZHprivateTax "Light"

The TAs will build a simplified 6-page clone of the **ZHprivateTax 2025** portal.

- **Pages:** Login, Personalien (Bio), Einkünfte (Income), Abzüge (Deductions), Vermögen (Wealth), and Review/Submit.
- **Behavior:** The UI does *not* have to be reactive. For example, in the official demo, selecting "Married" in the Bio page dynamically generates new fields

for a partner. In our simulator that will not be the case (too much development).

- **Knowledge Base:** A set of static, simplified HTML "Tax Guides" (Wegleitung) are hosted alongside the portal, covering rules for deductions (commuting, meals, Pillar 3a). These can be just a list of bullet points or sth like that.



Steueramt Kanton Zürich - Steuererklärung

ZHprivateTax 2025

https://zhp.services.zh.ch/app/ZHprivateTax2025/#/demo/tax-assistant/personal

4. The MCP Toolset

Students must interact with the environment exclusively through these tools. The agent has no "backdoor" to the data.

Browser & UI Tools (Discovery-Based)

- `scan_page()`: Returns a JSON representation of the current DOM. It lists all visible inputs, buttons, and text.
  - *Crucial:* It identifies elements by labels (e.g., "Bruttolohn") and provides a temporary `locator` string for the agent to use.

- `fill_field(locator, value)`: Enters text/numbers into a field discovered via `scan_page()`.
- `click_element(locator)`: Clicks buttons or links (e.g., "Next Page" or "Add Row").
- `submit_form()`: Submits the final return and generates the `submitted_return.json` for scoring.

Document & Knowledge Tools

- `list_documents()`: Lists the filenames in the persona's folder (e.g., `lohnuasweis.pdf`, `bank_statement.txt`).
- `read_document(filepath)`: Extracts raw text from `.txt` or text-based `.pdf` files. **No OCR is used.**
- `list_guides()`: Returns a list of available tax guide topics and their URLs.
- `fetch_guide(url)`: Returns the full text of a specific tax guide to help the agent understand deduction rules.

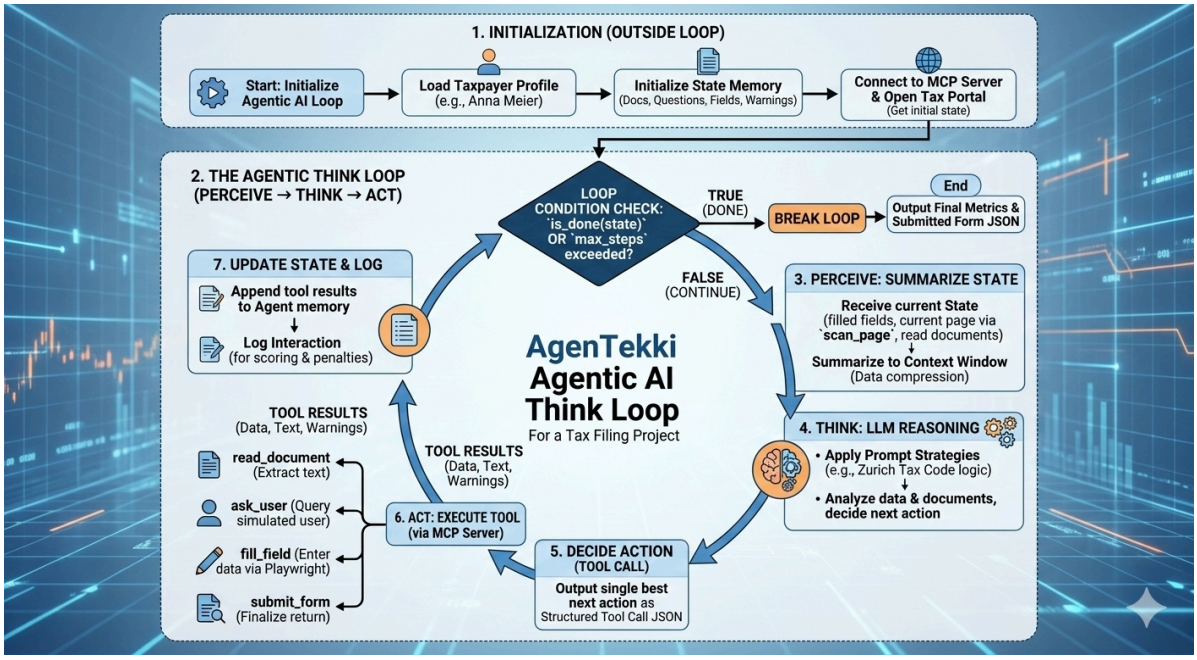
Human-in-the-Loop Tools

- `ask_user(question)`: Sends a query to the simulated taxpayer (an LLM NPC).
  - *Constraint:* Agents are penalized for asking questions if the answer was already present in the provided documents.

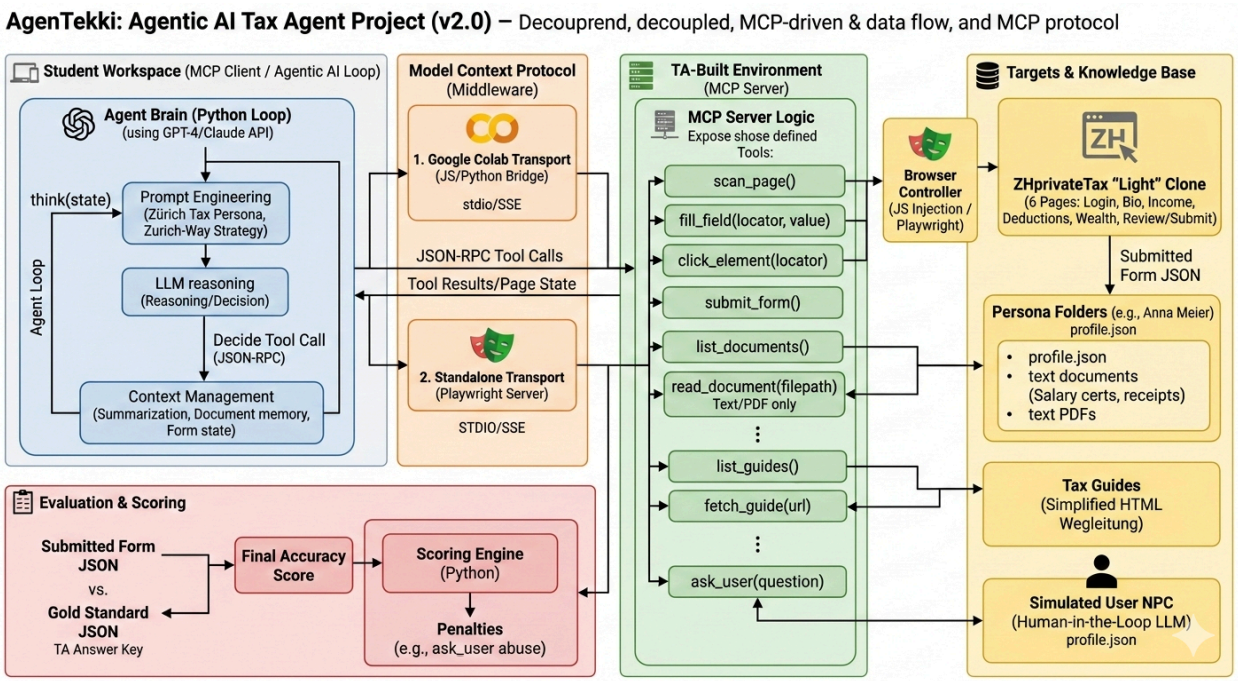
5. Participant Tasks

Students are responsible for the following development areas:

1. **MCP Server "Glue" (Partial):** Implementing the logic that maps an LLM's `fill_field` request to the specific browser-side execution (JS injection in Colab or Playwright command in Standalone).
2. **The Agentic Loop (Full):** Building the `think()` function that:
   - Analyzes the results of `scan_page()` to find where to put data.
   - Coordinates document reading with form filling.
   - Self-corrects based on validation errors returned by the UI.



3. **Prompt Engineering:** Creating the system instructions that create the agent, and that guide the agent through the specific "Zurich Way" of filing (e.g., "Always check the meal deduction if the taxpayer works in the city").



6. Development Plan (Decoupled Tracks)

To ensure the project can be built without a "dependency hell," TAs will work on these tracks in parallel:

- **Squad A: The Portal (Frontend) - TAs 1 & 2**
  - *Goal:* Build the 6-page ZHprivateTax React/HTML clone.
  - *Focus:* Clean semantic labels for `scan_page` discovery, form validation, and final JSON export .
- **Squad B: MCP & Automation (Middleware) - TAs 3 & 4**
  - *Goal:* Expose the UI and documents as MCP tools.
  - *Focus:* TA 3 builds the Standalone Playwright Server (for Claude Desktop/Gemini CLI). TA 4 builds the Colab JS-to-Python bridge. Both share the exact same MCP tool schemas
- **Squad C: Content & Personas (Data) - TAs 5 & 6**
  - *Goal:* Generate the "world".
  - *Focus:* TA 5 creates the 5 Personas (profiles, text documents, PDFs) and Gold Standard JSONs . TA 6 writes the simplified Tax Guides and sets up the `ask_user` NPC LLM logic.
- **Squad D: Integration & Evaluation (QA/Scoring) - TA 7**
  - *Goal:* Be the first "student".
  - *Focus:* Build the scoring script , assemble the Colab notebook scaffold, and write a baseline LLM `think()` loop to test the tools the moment they are ready .

7. Test Personas

The agent will be evaluated against these 5 standardized personas, ranging in difficulty:

Persona	Complexity	Key Challenge
Anna Meier	Easy	Single, one employer, basic deductions.
The Bernasconis	Medium	Married, dual income, childcare costs.

Priya Chakraborty	Hard	Foreign language documents, self-employment income.
The Widmers	Hard	Retirement mid-year, pension vs. salary split.
Yuki Tanaka	Expert	Divorced, alimony (taxable) vs. child support (non-taxable).

JSON Schema for `scan_page()`

```
{
  "page_name": "Income",
  "validation_errors": [],
  "elements": [
    {
      "type": "text",
      "content": "Please declare your employment income her
e."
    },
    {
      "type": "input",
      "locator": "field-income-employment-gross-salary",
      "label": "1. Gross Salary (CHF)",
      "value": "",
      "required": true
    },
    {
      "type": "select",
      "locator": "field-income-canteen",
      "label": "Subsidized canteen available?",
      "options": ["Yes", "No"],
      "value": "No",
      "required": true
    },
    {
      "type": "button",
      "locator": "btn-nav-deductions",
```

```
      "label": "Continue to Deductions"
    }
  ]
}
```

JSON example for form submission (generated by the page)

```
{
  "persona": "anna_meier",
  "pages": {
    "personal": {
      "full_name": "Anna Meier",
      "marital_status": "single"
    },
    "income": {
      "gross_salary": 142000,
      "social_security_contributions": 7526
    },
    "deductions": {
      "commuting_costs": 2840,
      "meal_allowance": 1600,
      "pillar_3a": 7056
    }
  }
}
```

Possible complete JSON Schema for form submission

```
{
  "persona": "string",
  "submitted_at": "datetime",

  "pages": {
    "personal": {
      "full_name": "string",
```

```
      "address": "string",
      "date_of_birth": "YYYY-MM-DD",
      "ahv_number": "string",
      "marital_status": "single | married | divorced | separa
ted | widowed",
      "profession": "string",
      "employer": "string",
      "workplace_city": "string",

      "partner": {
        "full_name": "string | null",
        "profession": "string | null",
        "employer": "string | null",
        "workplace_city": "string | null"
      },

      "children": [
        {
          "name": "string",
          "birth_year": "integer"
        }
      ]
    },

    "income": {
      "employment_gross_salary": "number | null",
      "employment_ahv_contributions": "number | null",
      "employment_bvg_contributions": "number | null",

      "self_employment_revenue": "number | null",
      "self_employment_expenses": "number | null",

      "pension_ahv": "number | null",
      "pension_bvg": "number | null",

      "capital_withdrawal": "number | null",
```

```
      "investment_dividends": "number | null",
      "investment_interest": "number | null",

      "alimony_received": "number | null",
      "other_income": "number | null",

      "real_estate_rental_value": "number | null"
    },

    "deductions": {
      "professional_expenses_type": "flat_rate | itemized",
      "professional_expenses_amount": "number | null",
      "commuting_costs": "number | null",
      "meal_allowance": "number | null",
      "pillar_3a_contributions": "number | null",
      "insurance_premiums": "number | null",
      "debt_interest": "number | null",
      "childcare_costs": "number | null",
      "further_education_costs": "number | null",
      "charitable_donations": "number | null",
      "medical_expenses": "number | null",
      "dual_income_deduction": "number | null",
      "property_maintenance": "number | null",
      "other_deductions": "number | null"
    },

    "wealth": {
      "securities": [
        {
          "name": "string",
          "isin": "string",
          "quantity": "number",
          "tax_value": "number",
          "gross_return": "number"
        }
```

```
      ],
      "bank_accounts": [
        {
          "name": "string",
          "balance_dec_31": "number",
          "interest": "number"
        }
      ],
      "real_estate_tax_value": "number | null",
      "other_assets": "number | null",

      "debts": [
        {
          "creditor": "string",
          "amount": "number"
        }
      ]
    },

    "attachments": [
      "string (file paths representing uploaded documents)"
    ]
  },

  "validation_warnings": [
    {
      "field_id": "string",
      "message": "string"
    }
  ]
}
```